

Salary Management System

Initial Requirements & Design Document

1. Objective

Build a web-based Salary Management System for ACME organization to replace the current Excel-based salary management process.

The system will enable an HR Manager to:

- Manage employee salary information.
- Manage salary components and salary adjustments.
- Maintain salary information across multiple countries and currencies.
- View salary history and effective-dated salary records.
- Search, filter, and explore employee compensation data.
- Analyze how the organization pays employees through salary analytics.
- View organization-level compensation metrics in a selected reporting currency.

The solution is designed to support approximately 10,000 employees.

2. User Persona

HR Manager

The primary user is an HR Manager responsible for managing and analyzing employee salary information across the organization.

Typical questions the system should help answer include:

- What is the average salary across the organization?
 - What is the median salary?
 - What is the total compensation?
 - Which country has the highest average compensation?
 - What is the average compensation for Engineering?
 - How does compensation differ by job level?
 - What is the salary of a particular employee?
 - How has an employee's salary changed over time?
-

3. Problem Statement

Currently, ACME's HR team manages salary information for approximately 10,000 employees across multiple countries using Excel spreadsheets.

This approach creates challenges around:

- Searching and filtering large amounts of salary data.
- Maintaining consistent salary information.
- Managing different currencies.
- Tracking salary changes over time.
- Generating organization-level salary insights.
- Reducing manual calculations and spreadsheet dependency.

The proposed application will centralize this information into a structured web-based system.

4. Scope

4.1 Employee Management

The system will maintain:

- Employee ID
- Employee name
- Email
- Country
- Department
- Job level
- Job title
- Employment status
- Hire date

The system will support approximately 10,000 seeded employee records.

4.2 Salary Management

The system will maintain salary records for employees.

A salary record will contain:

- Employee
- Salary currency
- Effective-from date
- Effective-to date
- Salary status
- Salary components

Salary records will be effective-dated so historical salary information is not overwritten.

4.3 Salary Components

Salary will be represented using configurable components.

Examples:

Earnings

- Basic Salary
- HRA
- Bonus
- Allowance
- Overtime
- Comp-Off Credit

Deductions

- Unpaid Leave
- Other Salary Deduction
- Recovery

Each component will have:

- Component code
- Component name
- Component type
- Calculation method
- Taxable flag
- Active/inactive status

Component types:

EARNING
DEDUCTION

Calculation methods:

FIXED
PERCENTAGE
PER_UNIT

This provides flexibility for salary-impacting adjustments without implementing a complete payroll engine.

5. Salary Calculation Approach

The system will not store a manually entered total compensation as the primary source of truth.

Instead, compensation will be derived from salary components.

Example:

Basic Salary	₹1,500,000
HRA	₹300,000
Bonus	₹150,000
Overtime	₹50,000

Total Earnings	₹2,000,000
Unpaid Leave	₹20,000

Adjusted Compensation	₹1,980,000

The calculation will be handled in the backend service layer.

The database will store the underlying salary components, while derived values such as total earnings and adjusted compensation will be calculated by the application.

This avoids inconsistent manually maintained totals.

6. Multi-Currency Support

Since employees belong to different countries, salary will be stored together with its original currency.

Example:

```
EMP001 → 1,500,000 INR
EMP002 → 100,000 USD
EMP003 → 80,000 EUR
EMP004 → 70,000 GBP
```

The system will maintain a currency master containing supported currencies.

Exchange rates will be obtained from an external currency-rate provider and cached/stored locally for reporting.

For organization-wide analytics, the HR Manager will be able to select a reporting currency.

Example:

```
Employee Salary
  ↓
Original Currency
  ↓
Exchange Rate
  ↓
Reporting Currency
  ↓
Organization Analytics
```

This allows metrics such as average, median, and total compensation to be calculated in a common currency.

For the assessment, live exchange-rate integration is not considered a core business requirement. Exchange rates may be synchronized/cached and used as the source for reporting.

7. Salary History

Salary records will use effective dates.

Example:

```
Employee: EMP00001

01-Apr-2024
Base Salary = ₹1,000,000

01-Apr-2025
Base Salary = ₹1,250,000

01-Apr-2026
Base Salary = ₹1,500,000
```

The currently active salary will be identified by the effective date/status.

Historical records will remain available for analysis and auditing.

8. Search and Filtering

The employee salary interface will support:

- Employee search
- Country filter
- Department filter
- Job level filter
- Job title filter
- Salary range filter
- Currency filter
- Employment status filter
- Sorting
- Pagination

Filtering and pagination will be performed server-side to avoid loading all 10,000 employees into the browser.

9. Salary Analytics

The dashboard will provide organization-level and filtered salary insights.

Organization metrics

- Employee count
- Average compensation
- Median compensation
- Minimum compensation
- Maximum compensation
- Total compensation

Breakdown

Analytics can be grouped by:

- Country
- Department
- Job level
- Job title

Example:

Average Compensation by Country

India	₹18.2L
-------	--------

USA	₹92K
Germany	€84K
UK	£72K

When comparing multiple currencies, values will first be normalized to the HR-selected reporting currency.

10. Proposed Database

The initial database will contain the following core entities:

```
countries
currencies
departments
job_levels
employees
salary_components
salary_records
salary_record_components
exchange_rates
```

Relationship overview:

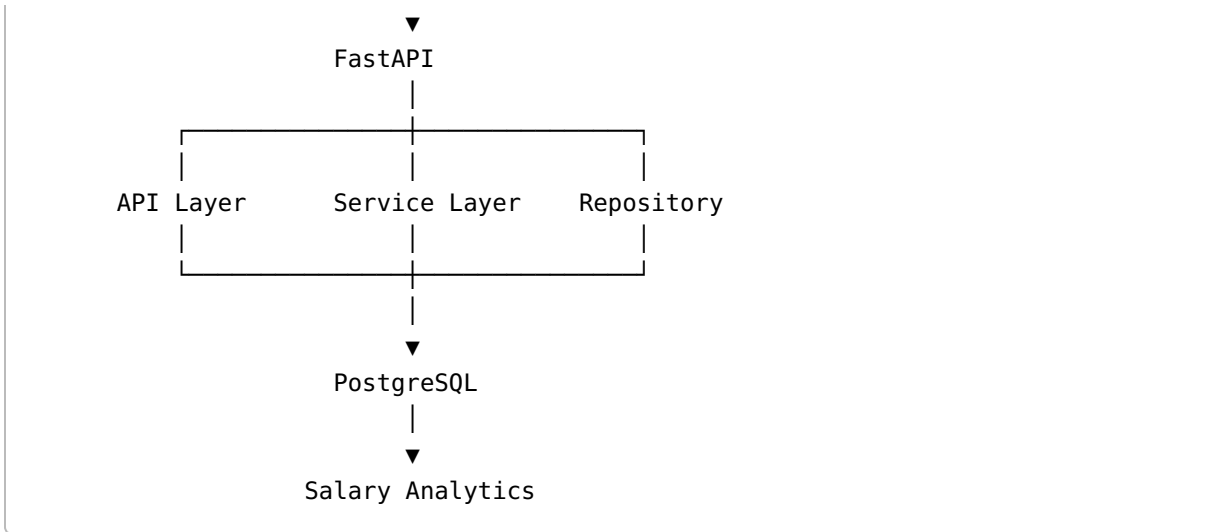
```
Country
  ↓
Employee
  ↓
Salary Record
  ↓
Salary Record Components
  ↓
Salary Components
```

Currency and exchange-rate data will support multi-country salary reporting.

11. High-Level Architecture

The application will initially use a modular monolithic architecture.

```
React / NextJS
  |
  | REST API
```



A modular monolith is preferred over microservices because the assessment does not require distributed services and the expected dataset is approximately 10,000 employees.

This reduces operational complexity while keeping the codebase modular and maintainable.

12. Data Seeding

A deterministic seed script will generate approximately 10,000 employees.

The seed data will include:

- Employees
- Countries
- Currencies
- Departments
- Job levels
- Job titles
- Salary records
- Salary components
- Salary amounts
- Salary effective dates

The generated data will be realistic enough to demonstrate:

- Search
- Filtering
- Pagination
- Salary distribution
- Country-level analytics
- Department-level analytics

- Job-level analytics
- Currency conversion

A fixed random seed will be used so that the generated dataset is deterministic and reproducible.

13. Testing Strategy

The application will include automated tests covering core business functionality.

Tests will cover:

- Employee creation and validation
- Salary record creation
- Salary component validation
- Salary calculation
- Earnings calculation
- Deduction calculation
- Salary history
- Currency conversion
- Employee filtering
- Pagination
- Salary analytics
- API validation and error handling

Tests should be fast, deterministic, isolated, and easy to understand.

14. Performance Considerations

The target dataset is approximately 10,000 employees.

The system will therefore use:

- Database indexes on frequently filtered columns.
- Server-side pagination.
- Server-side filtering and sorting.
- Efficient aggregation queries.
- Database-level constraints.
- Appropriate joins and query optimization.
- Cached exchange rates where appropriate.

The application will avoid loading the entire employee dataset into the frontend.

15. Security Considerations

The initial implementation will follow standard backend security practices:

- Input validation.
 - Parameterized database queries through the ORM.
 - Environment-based configuration.
 - No secrets committed to Git.
 - Appropriate API error handling.
 - Authentication/authorization where required.
 - Restricted salary-data access to authorized HR users.
-

16. Deliberately Out of Scope

The following features are deliberately excluded from the initial scope:

Full Payroll Processing

- Attendance processing
- Leave management
- Payroll periods
- Tax engine
- PF/ESI/statutory calculations
- Bank payment processing
- Salary transfer
- Payroll reconciliation

Other HRMS Modules

- Recruitment
- Performance management
- Employee onboarding
- Project management
- Training
- Benefits management

Advanced Integrations

- Banking APIs
- Accounting/ERP integrations
- Live payroll providers
- Real-time HRIS synchronization

Reason

The assessment focuses on managing salary data and enabling HR to understand organizational compensation rather than building a complete payroll or HRMS platform.

Keeping these capabilities outside the MVP allows the implementation to focus on correctness, maintainability, usability, scalability, testing, and product value.

17. Key Assumptions

1. The primary user is an HR Manager.
 2. The initial target is approximately 10,000 employees.
 3. Salary values are maintained in the employee's original currency.
 4. Exchange rates are used only for normalized reporting.
 5. Salary records are effective-dated.
 6. Salary totals are derived from salary components rather than manually maintained.
 7. Exchange-rate data can be cached/stored locally for predictable reporting.
 8. The initial system does not execute actual salary payments.
 9. The initial system is not a complete statutory payroll engine.
 10. The system is designed so additional payroll capabilities can be added later without redesigning the core salary data model.
-

18. Success Criteria

The solution will be considered successful if an HR Manager can:

1. Search and view employee salary information.
 2. Add and update salary records.
 3. Maintain salary history.
 4. Manage configurable salary components.
 5. Represent salary earnings and deductions.
 6. Filter employees by relevant organizational and salary attributes.
 7. Analyze compensation across countries, departments, and job levels.
 8. View organization-wide salary metrics in a selected reporting currency.
 9. Work efficiently with approximately 10,000 employees.
 10. Use the application through a deployed web interface.
 11. Trust the results through automated tests and deterministic data.
-

19. Implementation Approach

Development will be incremental:

1. Requirements & design
- ↓
2. Database schema & migrations
- ↓
3. Seed data generation
- ↓
4. Employee APIs
- ↓
5. Salary management
- ↓
6. Salary calculation
- ↓
7. Currency conversion
- ↓
8. Analytics APIs
- ↓
9. React/NextJS UI
- ↓
10. Tests
- ↓
11. Performance optimization
- ↓
12. Deployment
- ↓
13. Documentation & demo

Each major stage will be committed separately to Git so the repository history demonstrates the evolution of the solution.

20. Future Extension

The core salary model is designed so that future payroll functionality can be introduced without changing the fundamental employee/salary structure.

Potential future modules could include:

- Salary Management
- ↓
- Payroll
- ↓
- Tax & Statutory Compliance
- ↓
- Payment Processing



Accounting Integration

These capabilities are intentionally not part of the initial assessment scope.